11 Building tool-based agents with LangGraph

This chapter covers

- Building LLM-powered agents using LangGraph
- Registering and using tools for dynamic agent execution
- Debugging agent execution and tool calls
- Simplifying agents with prebuilt LangGraph components
- Observing agent execution with LangSmith

In chapter 5, we explored the distinction between agentic workflows and agents. You learned that agentic workflows are fundamentally deterministic: their logic is based on flows with conditional paths that depend on the current application state. These workflows can be elegantly modeled using node-based graphs in LangGraph, and you saw a complete, hands-on example of such a system.

Agents, however, operate differently. Rather than following a predetermined flow, agents rely on dynamic, context-sensitive decision-making. With the help of a large language model (LLM), an agent chooses which tools to use—and in what

order—based on the evolving context of the task at hand. These decisions aren't prescripted; instead, they unfold step-by-step, as the agent continually evaluates the outputs of previous actions and adapts accordingly.

In this chapter, you'll put these ideas into practice by building a multi-tool travel information agent. You'll begin simply, implementing an agent that provides destination information using a single tool. From there, you'll extend it into a true multi-tool agent, able to answer questions about both travel destinations and their current weather conditions.

As we progress, I'll introduce you to the core concepts necessary for constructing multi-tool agents, with particular focus on the tool-calling protocol. You'll first implement this protocol from scratch to grasp every detail, and then you'll see how LangGraph's built-in capabilities can streamline and simplify your agent's architecture.

This chapter's multi-tool agent will serve as the foundation for the more advanced, multi-agent systems you'll build in the chapters ahead. Let's dive in—there's a lot to discover.

11.1 Starting simple: Building a single-tool travel info agent

In this section, we'll lay the groundwork for our agent-based applications by building a straightforward travel information agent. This first agent will use just one tool: a vector store retriever that answers questions about Cornwall's destinations and resorts, using content from Wikivoyage (www.wikivoyage.org). The content is split into chunks and stored in a vector store for efficient retrieval.

If you've followed the advanced Retrieval-Augmented Generation (RAG) chapters earlier in this book, you should already be comfortable with sourcing content and populating a vector store. Here, we'll build on that foundation, keeping the focus on agent mechanics.

11.1.1 Project setup

Let's set up a new Python project using Visual Studio Code (VS Code). This works seamlessly with Cursor as well.

CREATING A VIRTUAL ENVIRONMENT AND INSTALLING DEPENDENCIES

First, set up your Python virtual environment and install all necessary dependencies. You'll find the `requirements.txt` file either on the Manning website for this book or in the cloned GitHub repository that accompanies chapter 11.

Open a new PowerShell terminal in VS Code (choose Terminal > New Terminal), navigate to the `ch11` project folder, and then create and activate a new virtual environment:

```
PS C:\Github\building-llm-applications\ch11> python -m venv env_ch11
PS C:\Github\building-llm-applications\ch11> .\env_ch11\Scripts\activate
(env_ch11) PS C:\Github\building-llm-applications\ch11>
```

Once your environment is activated, install the required dependencies:

```
(env_ch11) PS C:\Github\building-llm-applications\ch11>
➥ pip install -r .\requirements.txt
```

Your project environment is now ready for development.

ADDING YOUR OPENAI API KEY

Create an `.env` file in your project root, and add your OpenAI API key:

```
OPENAI_API_KEY=<Your OPENAI_API_KEY>
```

CONFIGURING VS. CODE DEBUGGING

For smooth debugging, add the following `launch.json` to your `.vscode` directory:

```
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Python Debugger: Current File",
            "type": "debugpy",
            "request": "launch",
            "program": "${file}",
            "console": "integratedTerminal"
        }
    ]
}
```

Organize your implementation files by using the naming convention `main_x_y.py` for the scripts. The `x` represents the feature (e.g., `main_01_01.py` for the initial travel info agent, `main_02_01.py` when adding weather), while `y` is the version of that feature as we iterate. This will make it easy for you to compare versions and follow the progression of the implementation.

> **NOTE** The code in these examples is intentionally simplified to focus on core functionality. Error handling and defensive programming are omitted for clarity and learning purposes.

11.1.2 Loading environment variables

Once your `.env` file is ready, create a code file named `main_01_01.py`. You can load your API key at the top of the script, after the import statements (omitted here for brevity—refer to the GitHub repository for the full list), as shown in the following listing.

Listing 11.1 Loading environment variables

```
load_dotenv()
```

Loads environment variables from the .env file

11.1.3 Preparing the travel information vector store

To enable our travel information agent to answer queries about Cornwall destinations, we first need a way to store and efficiently retrieve relevant information. The approach here draws on techniques you saw in chapter 8, sections 8.3 and 8.4, on advanced RAG techniques, but streamlines them for this agent-centric context. At a high level, we'll download travel content for a set of Cornwall destinations from Wikivoyage, break the

text into manageable chunks, embed those chunks into vector representations, and then store everything in a Chroma vector store. We'll also encapsulate the initialization logic in a singleton pattern to ensure that the vector store is only built once during the agent's lifetime. The following code sets up this vector store, making it easy to retrieve relevant travel information as the agent operates.

Listing 11.2 Preparing the travel information vector store

```
UK_DESTINATIONS = [
    "Cornwall",
    "North_Cornwall",
    "South_Cornwall",
    "West_Cornwall",
]

async def build_vectorstore(
    destinations: Sequence[str]) -> Chroma:
    """Download Wikivoyage pages and create
    a Chroma vector store."""
    urls = [f"https://en.wikivoyage.org/wiki/{slug}"
   for slug in destinations]
    loader = AsyncHtmlLoader(urls)
    print("Downloading destination pages ...")
    docs = await loader.aload()

    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1024, chunk_overlap=128)
    chunks = sum([splitter.split_documents([d])
        for d in docs], [])

    print(f"Embedding {len(chunks)} chunks ...")
    vectordb_client = Chroma.from_documents(
        chunks, embedding=OpenAIEmbeddings())
    print("Vector store ready.\n")
    return vectordb_client

_ti_vectorstore_client: Chroma | None = None

def get_travel_info_vectorstore() -> Chroma:
    global _ti_vectorstore_client
    if _ti_vectorstore_client is None:
        if not os.environ.get(
            "OPENAI_API_KEY"):
            raise RuntimeError(
                """Set the OPENAI_API_KEY env
                variable and re-run.""")
        _ti_vectorstore_client = asyncio.run(
            build_vectorstore(UK_DESTINATIONS))
    return _ti_vectorstore_client

ti_vectorstore_client
➥= get_travel_info_vectorstore()
ti_retriever = ti_vectorstore_client.as_retriever()
```

List of target Cornwall destinations—expand as needed

Asynchronous function to build and return the vector store

Downloads Wikivoyage content for each destination

Splits downloaded documents into manageable chunks

Embeds each chunk and stores it in the Chroma vector store

Returns the initialized vector store

Singleton cache for the vector store client

Function to initialize/retrieve the cached vector store

Returns the cached vector store client instance

Creates the vector store client

Creates a retriever from the vector store

This setup starts by defining a list of Cornwall-related destinations that serve as the basis for information retrieval. The `build_vectorstore()` asynchronous function constructs URLs for each destination and uses an asynchronous loader to fetch the corresponding Wikivoyage pages. Once the pages are downloaded, the text is split into overlapping chunks to ensure that information remains contextually meaningful. These chunks are then embedded using OpenAI's embedding models and stored in a Chroma vector store, making them quickly searchable by semantic similarity.

To prevent unnecessary recomputation and data downloads, the singleton pattern is used for the vector store client. The `get_travel_info_vectorstore()` function ensures that the vector store is only built once and is reused for all future retrievals. At the end, the vector store client is instantiated, and a retriever object is created from it—this retriever will be the agent's interface for accessing Cornwall travel information. This foundation allows the agent to efficiently answer user queries about destinations using up-to-date knowledge sourced directly from Wikivoyage.

11.2 Enabling agents to call tools

Now that we have our vector store retriever ready, the next step is to expose this retrieval capability as a tool the agent can use. This is where the concept of tool calling—a major advance in modern agent frameworks—enters the picture.

11.2.1 From function calling to tool calling

LLMs like those from OpenAI initially introduced *function calling*—a mechanism that allows the model to request specific functions, passing structured arguments, based on the needs of a given prompt. This concept quickly evolved into the more general *tool-calling* protocol, now widely supported by major LLM providers. With tool calling, models can invoke not just custom functions but also a variety of built-in or external "tools," handling everything from code execution to external API lookups.

> **DEFINITION** A *tool* is any external function, service, or capability that an LLM can call through the tool-calling protocol. Tools extend the model's abilities beyond text generation—for example, running code, querying a database, or calling an API—by letting the model pass structured inputs and receive structured outputs.

You might wonder why tool calling matters. Why not just let agents access functionality directly through standard REST API endpoints?

The key reason is flexibility. We turn to agents—rather than fixed agentic workflows—when we can't predict in advance which tools will be needed, in what sequence they should be called, or how a user's question will map to tool parameters. In these cases, hardcoding API calls isn't enough.

An agent's strength lies in making those decisions dynamically: selecting the right tool, determining the correct order of calls, and shaping inputs to match tool parameters. Modern LLMs are explicitly trained for this purpose, with tool calling built into their response protocols (e.g., the OpenAI Responses API). In short, tool calling unlocks the adaptive reasoning that makes multi-tool agents possible.

This evolution has made agent implementations dramatically simpler and more robust, particularly when using the *ReAct* (Reasoning and Acting) design pattern. The ReAct pattern, introduced by Yao et al. in their 2022 paper "ReAct: Synergizing Reasoning and Acting in Language Models" (https://arxiv.org/abs/2210.03629) enables LLMs to interleave reasoning steps ("thoughts") with tool invocations ("actions"). In effect, the model can break a task into smaller pieces, use tools where appropriate, and synthesize an answer step-by-step. Figure 11.1 shows a simplified view of the ReAct pattern as a process diagram.

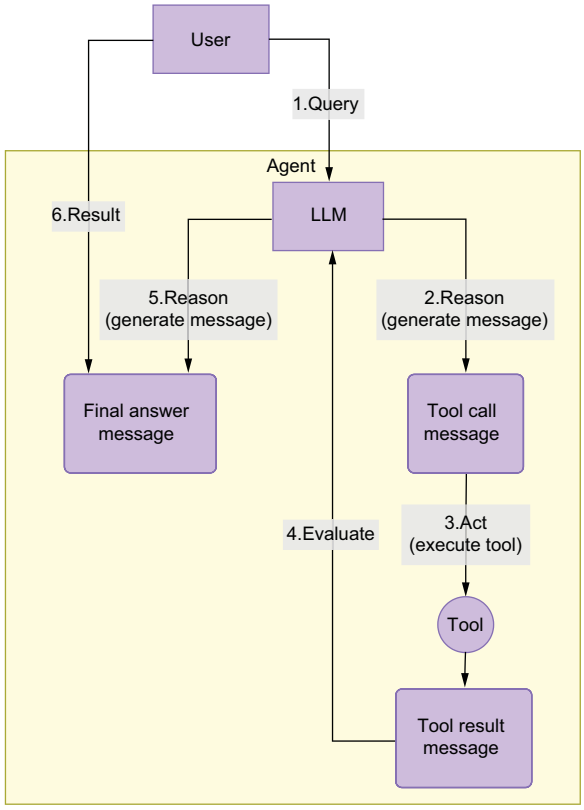

Figure 11.1 The ReAct pattern alternates between reasoning and action, enabling the agent to process a user question by thinking, calling tools as needed, and following up with further reasoning before delivering the final answer.

The ReAct pattern starts with a user question, as shown in the diagram. The agent first enters the Reasoning phase, where the LLM analyzes the input and decides on the next steps. If further information or actions are required, the agent moves into the Act phase, calling one or more tools—such as running a semantic search or invoking an API.

Once the tool returns results, the agent cycles back into Reasoning, incorporating the new information into its thought process. This may trigger additional tool calls, or the agent may determine that it has enough context to answer. The process concludes when the agent produces a Final Answer for the user.

This stepwise interplay between reasoning and acting embodies the core of the ReAct pattern, allowing agents to dynamically combine thought and action as they work toward solving the user's query.

> **NOTE** With the introduction of the OpenAI Responses API, the transition from function calling to tool calling has accelerated. The Responses API is designed specifically for structured tool use and has largely superseded the older Completion API for agentic applications.

11.2.2 How tool calling works with LLMs

OpenAI's tool calling supports several types of tools—including user-defined functions, code interpreters, and native capabilities such as web browsing. At a high level, when you register a tool with the LLM, you expose both the function signature (name and parameters) and a textual description. The model then decides, at runtime, when and how to use each tool based on the user's input and the context of the conversation.

In LangGraph, tools can be registered using either a class-based or a decorator-based approach. We'll use the decorator-based approach for clarity and conciseness. Let's implement our semantic search tool as a function decorated with `@tool`, making it available to the agent for tool calling, as shown in the following listing.

Listing 11.3 LangGraph attribute-based tool definition

```
@tool
def search_travel_info(query: str) -> str:
    """Search embedded Wikivoyage content for
    information about destinations in England."""
    docs = ti_retriever.invoke(query)
    top = docs[:4] if isinstance(docs, list) else docs
    return "\n---\n".join(
        d.page_content for d in top)
```

Defines the tool using the @tool decorator

Defines the tool function, which takes a query, performs a semantic search, and returns a string response from the vector store

Performs a semantic search on the vector store and returns the top four results

Joins the top four results into a single string

This decorated function, `search_travel_info()`, is now recognized as a tool: it takes a user query, searches the vector store for relevant Wikivoyage content, and returns up to four top results as a single string. The `@tool` decorator ensures that the function's name, description, and parameter schema are all available to the LLM for tool calling.

11.2.3 Registering tools with the LLM

To enable the agent to use our semantic search tool, we must register it with the LLM so that the model knows both the function's signature and its intended purpose. In LangChain, this is achieved using the `bind_tools` protocol, but it's instructive to see how this works at the OpenAI API level first.

With the introduction of the OpenAI Responses API, the way tools and functions are registered has become more standardized and explicit. Now you provide a

structured definition of your tool—specifying its name, description, and parameter schema. The model can then return responses that explicitly request tool invocations, passing arguments as needed. For further details, you can consult the OpenAI function calling documentation (https://mng.bz/Bzag).

EXAMPLE: MANUAL TOOL REGISTRATION WITH OPENAI API

Suppose you wanted to expose our `search_travel_info` function directly to the OpenAI API (without using LangChain). You would define the tool's schema as shown in listing 11.4. Note that this example is for illustration only and isn't included in the book's provided source code.

Listing 11.4 Manual tool registration with the OpenAI API

```
search_travel_info_tool = {
    "type": "function",
    "function": {
        "name": "search_travel_info",
        "description": "Search embedded Wikivoyage content for information
        ➥about destinations in England.",
        "parameters": {
            "type": "object",
            "properties": {
                "query": {
                    "type": "string",
                    "description": "A natural language
                    ➥ query about a destination in
                    ➥ England."
                }
            },
            "required": ["query"]
        }
    }
}

response = client.chat.completions.create(
    model="gpt-5",
    messages=[
        {"role": "user", "content": "Tell me about surfing in
        ➥Cornwall."}
    ],
    tools=[search_travel_info_tool],
    tool_choice="auto",
)
```

Specifies that this object is a function tool (as per OpenAI's tool-calling schema)

Name of the tool/function being registered

Human-readable description of what the tool does

The parameters accepted by the function are described in this dictionary.

The parameters are passed as an object (i.e., a dictionary of named arguments).

The function accepts a single parameter named "query."

Description of what the "query" parameter should contain

"query" is a required argument for the tool.

Specifies the OpenAI model to use for the completion

Provides the initial user message to the chat (the question to answer)

Registers the tool so the model knows it can call it if needed

Allows the model to automatically decide whether and when to use the tool

In this example, you explicitly define the tool's metadata and parameters and then pass it in the `tools` argument of your API call. When the model decides it needs to call `search_travel_info`, it will return a structured tool call in its response. Your application must then handle the invocation of the Python function, passing the model-generated arguments, and send the results back to the LLM if the conversation continues.

REGISTERING TOOLS IN LANGCHAIN

LangChain automates much of this process. You simply define your tool as a decorated Python function (as you saw earlier in listing 11.3) and then register it with the LLM. You can see how this looks in code in the following listing.

Listing 11.5 Registering tools in LangChain

```
TOOLS = [search_travel_info]

llm_model = ChatOpenAI(
    model="gpt-5-mini",
  use_responses_api=True)
llm_with_tools = llm_model.bind_tools(TOOLS)
```

Defines the tools list (in our case, only one tool)

Instantiates the LLM model with the GPT-5-mini model and the responses API

Binds the tools to the LLM model, which will generate a response with the tool calls

Here, we list the available tools (currently just `search_travel_info`), instantiate the `gpt-5-mini` chat model (with Responses API support), and use `.bind_tools(TOOLS)` to expose those tools to the model for tool calling.

LangChain handles the translation between your Python code and the OpenAI function/tool-calling protocol, including automatically generating the appropriate JSON schema from your function signature and docstring. This setup ensures that whenever the model receives a user message, it can autonomously decide if and when to call any registered tool, structuring its responses according to the tool-calling protocol.

> **NOTE** If you're using a model other than OpenAI, or you prefer not to use the Responses API, tool calling can still work—though the capabilities and structure of the responses may differ. The older Completion API is still available, but for most agentic use cases, the newer Responses API is recommended for its clarity, power, and alignment with evolving best practices.

11.2.4 Agent state: Tracking the conversation

In our implementation, the agent's state is simply a collection of LLM messages that track the entire conversation. This is defined as follows:

```
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
```

Here, `AgentState` only contains the sequence of messages exchanged between the user, the agent, and any tool responses. This keeps the design simple.

11.2.5 Executing tool calls

The core of the tool execution logic is implemented in a node that examines the LLM's most recent message, extracts any tool calls requested by the model, and invokes the corresponding functions with the provided arguments. Each tool's output is then wrapped in a message and appended to the conversation state. A simplified implementation is shown in the following listing.

Listing 11.6 Tool execution implemented from scratch

```
class ToolsExecutionNode:
    """Execute tools requested by the LLM in the last AIMessage."""

    def __init__(self, tools: Sequence):
        self._tools_by_name = {t.name: t for t in tools}

    def __call__(self, state: dict):
        messages: Sequence[BaseMessage] = state.get("messages", [])

        last_msg = messages[-1]
        tool_messages: list[ToolMessage] = []
        tool_calls = getattr(last_msg,
            "tool_calls", [])

        for tool_call in tool_calls:
            tool_name = tool_call["name"]
            tool_args = tool_call["args"]
            tool = self._tools_by_name[tool_name]
            result = tool.invoke(tool_args)
            tool_messages.append(
                ToolMessage(
                    content=json.dumps(result),
                    name=tool_name,
                    tool_call_id=tool_call["id"],
                )
            )
        return {"messages": tool_messages}

tools_execution_node = ToolsExecutionNode(TOOLS)
```

Defines the tools execution node

Initializes the tools execution node with the tools list

Defines the __call__ method, which is called when the node is invoked

Gets the last message from the messages list

Initializes the tool messages list to gather the results of the tool calls

Gets the tool calls from the last message

Iterates over the tool calls

Gets the tool name from the tool call

Gets the tool arguments from the tool call

Gets the tool from the tools list

Invokes the tool with the arguments

Adds the tool result to the tool messages list

Returns the tool messages list, which contains the results of the tool calls

Instantiates the tools execution node to be used as a node in the LangGraph graph

This pattern allows the agent to handle multiple tool calls in a single step. The `ToolsExecutionNode` inspects the LLM's latest response, invokes each requested tool by name, collects the results, and formats them for the next stage in the conversation. In a typical agent workflow, these tool results are sent back to the LLM, which incorporates the new information to either reason further or generate a direct answer to the user's query.

The built-in ToolNode class

You usually won't need to write this logic yourself in practice because LangGraph provides a built-in class, `ToolNode`, which performs the same function as our custom `ToolsExecutionNode`. For most applications, you can use

```
tools_execution_node = ToolNode(TOOLS)
```

This saves you from having to implement the tools execution logic manually, streamlining your agent development process.

Now that you understand how tool execution is managed, let's explore how the agent, guided by the LLM, determines which tools to call—or whether it already has enough information to generate the final answer.

11.2.6 The LLM node: Coordinating reasoning and action

Next, we add an LLM node to our LangGraph workflow, as shown in the following listing.

Listing 11.7 LLM node

```
def llm_node(state: AgentState):
    """LLM node that decides whether
    to call the search tool."""
    current_messages = state["messages"]
    response_message = llm_with_tools.invoke(
        current_messages)

    return {"messages": [response_message]}
```

Defines the LLM node

Gets the current messages from the agent state

Invokes the LLM model with the current messages. The LLM will decide whether to call the search tool or return an answer.

Returns the response message, which contains the tool call or the answer

This node takes the agent state (a list of messages) and forwards them to the LLM for processing. Because we're using an LLM with tool calling enabled (`llm_with_tools`), the model automatically understands when to issue tool calls and when to produce a final answer:

- If the last message is a user question, the LLM can decide to request a tool call (or multiple calls) to retrieve relevant information.
- If the last message(s) are tool results, the LLM integrates those responses, reasons further, and may either produce a final answer or request additional tool calls if needed.

The model returns either an `AIMessage` with a final answer in the content field, or additional tool calls in the `tool_calls` field, depending on the situation. This flexible, reactive loop is at the heart of all modern agent implementations, and it's what makes today's LLM-based agents so capable.

11.3 Assembling the agent graph

With our LLM node and tool execution node ready, the next step is to assemble them into a working agent graph. In LangGraph, an agent is structured as a directed graph, where each node represents a component (e.g., an LLM or a tool executor), and edges represent the possible flow of data and control between these nodes.

The code in listing 11.8 demonstrates how we assemble our single-tool travel information agent as a graph. Each node in the graph corresponds to either the LLM or the tool execution logic.

Listing 11.8 Graph of the tool-based agent

```
builder = StateGraph(AgentState)
builder.add_node("llm_node", llm_node)
builder.add_node("tools", tools_execution_node)

builder.add_conditional_edges("llm_node",
    tools_condition)

builder.add_edge("tools", "llm_node")

builder.set_entry_point("llm_node")
travel_info_agent = builder.compile()
```

Defines the graph builder

Adds the LLM node and the tools node to the graph

Adds the conditional edges to the graph to decide whether to execute the tool calls or return an answer and exit the graph

Adds the edge from the tools node to the LLM node

Sets the entry point to the LLM node

Compiles the graph

11.4 Understanding the agent graph structure

Our agent graph consists of two main nodes, as shown in figure 11.2: the LLM node, responsible for reasoning and generating tool calls, and the tools node, responsible for executing the requested tools and returning results. The flow of the conversation alternates between these two nodes, reflecting the ReAct pattern described earlier in this chapter.

A critical aspect of this setup is the conditional edge that connects the LLM node to the next step. This is controlled by the `tools_condition` function—a prebuilt utility in LangGraph. This function examines the latest message from the LLM:

- If the message contains the `tool_calls` property (meaning the LLM is requesting one or more tool invocations), the graph routes the flow to the node named `"tools"`.
- If there are no tool calls present, the flow is directed to the `END` node, terminating the graph and producing a final answer for the user.

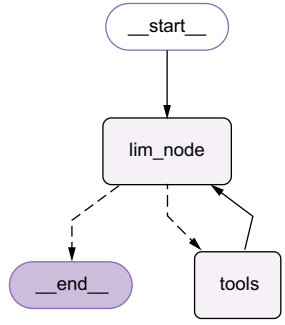

Figure 11.2 Conditional graph logic routes each user query through the LLM node and then dynamically directs flow to the tools node or ends the process, depending on whether tool calls are required.

This mechanism lets the agent dynamically decide at each turn whether to continue reasoning, take action, or conclude the interaction.

We explicitly set the entry point of the graph to `"llm_node"`, ensuring that each user question is first processed by the language model. (As an alternative, you could achieve the same effect by adding an edge from the `START` node to `"llm_node"` with `graph_builder.add_edge(START, "llm_node")`.) By compiling the graph with `builder.compile()`, we finalize our travel information agent, ready to receive queries and intelligently use its tool to find relevant travel information.

11.5 Running the agent chatbot: The Read-Eval-Print Loop

The final step in building our agent-powered chatbot is to implement the user interface—a simple loop that continuously accepts user questions and returns answers until the user chooses to exit. This classic pattern, known as a Read-Eval-Print Loop (REPL), is the main bridge between the user and your travel information agent.

At a high level, the chat loop listens for input, wraps the user's question in a message structure, invokes the agent graph, and prints the assistant's reply. This interaction continues indefinitely, enabling a true conversational experience. The following listing shows how you can implement this chat loop in Python.

Listing 11.9 Chatbot REPL

```
def chat_loop():
    print("UK Travel Assistant (type 'exit' to quit)")
    while True:
        user_input = input(
            "You: ").strip()
        if user_input.lower() in \
            {"exit", "quit"}:
                break
        state = {"messages":
            [HumanMessage(content=user_input)]}
        result = travel_info_agent.invoke(
            state)
        response_msg = result["messages"][-1]
        print(
           f"Assistant: {response_msg.content}\n")

if __name__ == "__main__":
    chat_loop()
```

Defines the chat loop

Gets the user input

Checks if the user input is "exit" or "quit" to exit the loop

Creates the initial state with a HumanMessage containing the user input

Invokes the graph with the initial state

Gets the last message from the result, which contains the final answer

Prints the assistant's final answer from the content of the preceding message

This loop welcomes the user, waits for their question, and continues processing until the user types `exit` or `quit`. Each input is packaged as a `HumanMessage` and passed to the travel information agent you just built. The agent's reply is extracted from the graph's output and displayed back to the user.

With this in place, you're now ready to run your first agent-based chatbot. Try asking it about destinations or activities in Cornwall, and experience how the agent reasons, retrieves information, and converses, all within the framework you've just constructed, as we're about to see.

11.6 Executing a request

Now that your travel information agent is ready, let's step through the agent in action by running and debugging your implementation. This hands-on walkthrough will show you how the agent orchestrates the flow between the LLM and the tool, helping you see the LangGraph framework in motion.

11.6.1 Step-by-step debugging

Begin by opening your `main_01_01.py` file and running it in debug mode, using the Python debug configuration you set up in your `launch.json` earlier. To trace the agent's flow, place breakpoints at the beginning of the following functions:

- `search_travel_info()`
- `ToolsExecutionNode.__call__()`
- `llm_node()`

Ready? Press F5 (or click the Play icon in your IDE), and let's walk through the agent's workflow together.

VECTOR STORE CREATION

When you start the script, you'll see the vector store being created and populated with travel information. In your debug console, you'll see the output similar to that in figure 11.3.

```
USER_AGENT environment variable not set, consider setting it to identify your requests.
Downloading destination pages ...
Fetching pages: 100%|#############################################################################################
#########| 4/4 [00:01<00:00,  3.96it/s]
Embedding 1118 chunks ...
Vector store ready.

UK Travel Assistant (type 'exit' to quit)
```

Figure 11.3 Output at startup during vector store creation

CHATBOT LOOP LAUNCH

Next, the chatbot loop starts up, waiting for your input:

```
UK Travel Assistant (type 'exit' to quit)
You:
```

Enter your question, and press Enter:

```
You: Suggest three towns with a nice beach in Cornwall
```

LLM NODE ACTIVATION

Your breakpoint inside `llm_node()` will trigger. Inspect `state["messages"]`:

```
HumanMessage (content='Suggest three towns with a nice beach in Cornwall',
additional_kwargs={}, response_metadata={})
```

Step over (press F10) to the next line to send this message to the LLM. Because the LLM is configured for tool calling, examine the resulting `response_message`:

```
   AIMessage(content=[],
...
tool_calls=[
```

```
{'name': 'search_travel_info', 'args': {'query': 'beach towns in Cornwall'},
'id': 'call_L4PwmeyLkrkYX2PfC2gY6ri6', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'best beaches in Cornwall'},
'id': 'call_XPNctNtyIKVetJa3ruM9z7d5', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'top seaside towns
Cornwall'}, 'id': 'call_RAWMLwdFVALuIbJxWKfta5yp', 'type': 'tool_call'}],
...)
```

Here, the LLM has generated three tool calls—each targeting your semantic search tool with a slightly different query. Notice how the model rewrites the queries to maximize information coverage, as discussed in chapter 9. You don't need to manually handle query rewriting—the LLM handles it.

TOOLS EXECUTION NODE

Continue execution (press F5). The list of tool calls is added to the message list. The conditional edge's `tools_condition` detects tool calls, routing execution to `ToolsExecutionNode.__call__()`. Your breakpoint will trigger there.

Next, inspect `state["messages"]`:

```
[
HumanMessage(content='Suggest three towns with a nice beach in Cornwall',
additional_kwargs={}, response_metadata={}),
AIMessage(content=[], ... ,
tool_calls=[
{'name': 'search_travel_info', 'args': {'query': 'beach towns in Cornwall'},
'id': 'call_L4PwmeyLkrkYX2PfC2gY6ri6', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'best beaches in Cornwall'},
'id': 'call_XPNctNtyIKVetJa3ruM9z7d5', 'type': 'tool_call'},
{'name': 'search_travel_info', 'args': {'query': 'top seaside towns
Cornwall'}, 'id': 'call_RAWMLwdFVALuIbJxWKfta5yp', 'type': 'tool_call'}] ,
...)
]
```

The last message (from the LLM) has no content (the LLM hasn't answered yet because it needs more information), but it contains the tool calls that must be executed. The node extracts these, then iterates through each one, extracting the tool name and arguments, and invokes the corresponding tool:

```
result = tool.invoke(tool_args)
```

Step through this to watch `search_travel_info()` execute. Each semantic search result (a document returned from the vector store) is collected as a `ToolMessage` and added to the list for the next LLM step.

TOOL CALL RESULTS PASSED BACK TO LLM

Continue (press F5), and you'll return to `llm_node()`. Now `state["messages"]` contains your original question, the LLM's tool call instructions, and the results from each tool execution:

```
\[HumanMessage(content='Suggest three towns with a nice beach in
Cornwall', additional\_kwargs={}, response\_metadata={}),
```

```
   AIMessage(content=\[], ...
   tool\_calls=\[{'name': 'search\_travel\_info', 'args': {'query': 'beach
   towns in Cornwall'}, ...}, ...]),
   ToolMessage(content='...', name='search\_travel\_info',
   tool\_call\_id='call\_L4PwmeyLkrkYX2PfC2gY6ri6'),
   ToolMessage(content='...', name='search\_travel\_info',
   tool\_call\_id='call\_XPNctNtyIKVetJa3ruM9z7d5'),
   ToolMessage(content='...', name='search\_travel\_info',
   tool\_call\_id='call\_RAWMLwdFVALuIbJxWKfta5yp')]
```

The content of each tool message gives the LLM the facts it needs to synthesize a final answer. Step over the following line:

```
response\_message = llm\_with\_tools.invoke(current\_messages)
```

Inspect `response_message`:

```
AIMessage(content=\[{'type': 'text', 'text': "Three towns in Cornwall with
nice beaches are:\n\n1. Newquay - Known as the UK's surfing capital with
popular beaches like Fistral Beach.\n2. St Ives - A picturesque town with
beautiful sandy beaches.\n3. Falmouth - Located on the south coast with
beaches like Gyllyngvase beach.\n\nThese towns offer great beach experiences
along with other attractions.", 'annotations': \[]}], ...)
```

Now the `content` field is populated with the LLM's answer. The `tool_calls` field is gone—the LLM no longer needs external tools and has synthesized a response.

COMPLETING THE REQUEST

As execution leaves `llm_node()`, the `tools_condition` on the conditional edge checks for further tool calls. Finding none, it ends the conversation. When your main invocation returns

```
result = travel\_info\_agent.invoke(state)
```

the `result.messages` list concludes with the final `AIMessage` containing the answer:

```
   {'messages': [
   HumanMessage(content='Suggest three towns with a nice beach in Cornwall',
...),
   AIMessage(content=[],  tool_calls=[...]),
   ToolMessage(...), ToolMessage(...), ToolMessage(...),
   AIMessage(content=[{'type': 'text', 'text': "Three towns in Cornwall with
nice beaches are:\n\n1. Newquay - Known as the UK's surfing capital with
popular beaches like Fistral Beach.\n2. St Ives - A picturesque town with
beautiful sandy beaches.\n3. Falmouth - Located on the south coast with
beaches like Gyllyngvase beach.\n\nThese towns offer great beach experiences
along with other attractions.", 'annotations': []}], ...)
   ]
```

the chatbot will now display the answer and prompt for your next question:

```
UK Travel Assistant (type 'exit' to quit)
You: Suggest three towns with a nice beach in Cornwall
Assistant: [{'type': 'text', 'text': "Three towns in Cornwall with nice
beaches are:\n\n1. Newquay - Known as the UK's surfing capital with popular
```

```
beaches like Fistral Beach.\n2. St Ives - A picturesque town with beautiful
sandy beaches.\n3. Falmouth - Located on the south coast with beaches like
Gyllyngvase beach.\n\nThese towns offer great beach experiences along with
other attractions.", 'annotations': []}]

You:
```

You've now observed, step-by-step, how your agent processes a user request: interpreting the question, generating tool calls, executing them, and synthesizing a final, well-informed answer. This debug-driven walkthrough offers deep insight into the mechanics of agentic reasoning and action in LangGraph.

11.7 Expanding your agent: Adding a weather forecast tool

So far, our agent has been able to answer travel-related queries using semantic search over curated travel content. But real-world travel advice often depends on dynamic, real-time information—like the weather! In this section, you'll learn how to extend your agent by adding a second tool, enabling it to respond with context-aware answers based on current conditions.

11.7.1 Implementing a mock weather service

To illustrate the process, start by copying your `main_01_01.py` file to a new script called `main_02_01.py`. This will help you track each evolutionary step in your agent's development.

First, let's introduce a mock weather service. This service will simulate real-time weather data for any given town, returning both a weather condition (e.g., sunny or rainy) and a temperature. You can see the implementation of the `WeatherForecastService` in the following listing.

Listing 11.10 `WeatherForecastService`

```
class WeatherForecast(TypedDict):
    town: str
    weather: Literal["sunny", "foggy", "rainy", "windy"]
    temperature: int

class WeatherForecastService:
    _weather_options = ["sunny", "foggy", "rainy", "windy"]
    _temp_min = 18
    _temp_max = 31

    @classmethod
    def get_forecast(cls, town: str) \
        -> Optional[WeatherForecast]:

        weather = random.choice(cls._weather_options)
        temperature = random.randint(cls._temp_min, cls._temp_max)
        return WeatherForecast(town=town,
             weather=weather,
             temperature=temperature)
```

Defines the get_forecast method, which returns a WeatherForecast object

This mock service chooses a random weather condition and temperature within a typical summer range for Cornwall. Later, you can swap this out for a real-world weather API if desired.

11.7.2 Creating the weather forecast tool

With the mock service in place, the next step is to create a tool that wraps it, making weather data available to the agent. Here's how you can define the tool function and add it to your agent's toolkit:

```
@tool
def weather_forecast(town: str) -> dict:
    """Get a mock weather forecast for a given town. Returns a
    ➥ WeatherForecast object with weather and temperature."""
    forecast = WeatherForecastService.get_forecast(town)
    if forecast is None:
        return {"error": f"No weather data available for '{town}'."}
    return forecast
```

The `weather_forecast` tool provides a mock weather forecast for a given town. When called with a town name, it returns a dictionary containing the weather conditions (e.g., sunny, foggy, rainy, or windy) and temperature for that location. If no data is available, it returns an error message instead. This tool allows the agent to incorporate simulated real-time weather information into its responses.

11.7.3 Updating the agent for multi-tool support

Finally, make sure your LLM model is set up to use tool calling and recognizes both available tools:

```
TOOLS = [search_travel_info, weather_forecast]

llm_model = ChatOpenAI(
    model="gpt-5-mini",
    use_responses_api=True
)
llm_with_tools = llm_model.bind_tools(TOOLS)
```

Defines the tools list (in our case, search_travel_info and weather_forecast)

Instantiates the LLM model with the GPT-5-mini model and enables the Responses API for tool calling

Binds the tools to the LLM model, enabling tool-calling support

With this setup, your agent is now equipped to handle both travel queries and weather checks, giving it the foundation to provide much more accurate and useful responses. In the next sections, you'll see how to guide the LLM to use both tools effectively, and you'll observe the agent's new capabilities in action. This workflow not only shows how to extend your agent's toolset but also demonstrates the modular, incremental approach that makes agentic systems with LangGraph and LangChain so powerful.

11.8 Executing the multi-tool agent

With your weather forecast tool registered, your agent can now synthesize answers that combine travel information and real-time weather conditions. The beauty of this

modular approach is that adding a new tool requires no changes to your agent's graph structure. All orchestration is handled by the LLM and the tool-calling protocol.

11.8.1 Running the multi-tool agent (initial behavior)

Let's put the new capabilities to the test. Set the same breakpoints as before—especially in the tool execution and LLM nodes—and add a breakpoint at the start of `weather_forecast()`. Run `main_02_01.py` in debug mode (press F5), and enter the following prompt:

```
You: Suggest two Cornwall beach towns with nice weather
```

After submitting your query, your first breakpoint will hit in `llm_node()`. Step through to the last line, and inspect the value of `response_message.tool_calls`:

```
[{'name': 'weather_forecast', 'args': {'town': 'Newquay'}, 'id':
'call_3nIgMLIFgMZBvVvrTHbRh2lj', 'type': 'tool_call'},
{'name': 'weather_forecast', 'args': {'town': 'Falmouth'}, 'id':
'call_Qfv5ENGOXhEUUcAAEamDyoQU', 'type': 'tool_call'}]
```

Here, the LLM has simply picked two towns it "knows" (Newquay and Falmouth) and asked for their weather (you might get a different combination of towns), without consulting your semantic search tool at all. This is typical of a model using its internal knowledge base rather than the tools provided—something we want to avoid for reliability and accuracy.

Why does this happen? The LLM is acting on its pretraining and defaulting to what it already "knows" about Cornwall, rather than querying your up-to-date data.

11.8.2 Improving LLM tool usage with system guidance

To nudge the LLM toward tool use and away from hallucinations, let's make its instructions and tool descriptions clearer. Make a copy of your script as `main_02_02.py`, and update the tool definitions by adding a description of each tool:

```
@tool(description="""Search travel information
about destinations in England.""")
def search_travel_info(query: str) -> str:
    ...

@tool(description="Get the weather forecast, given a town name.")
def weather_forecast(town: str) -> dict:
    ...
```

Next, introduce a guiding `SystemMessage` in your `llm_node()`:

```
system_message = SystemMessage(content="""You are a helpful assistant
    that can search travel information and get the weather forecast.
    Only use the tools to find the information
    you need (including town names).""")
current_messages.append(system_message)
```

You can see the amended `llm_node()` function in the following listing.

Listing 11.11 Guiding tool selection

```
def llm_node(state: AgentState):
    """LLM node that decides whether to call the search tool."""
    current_messages = state["messages"]
    system_message = SystemMessage(content="""You are a helpful assistant
    that can search travel information and get the weather forecast.
    Only use the tools to find the information
    you need (including town names).""")

    current_messages.append(system_message)
    response_message = llm_with_tools.invoke(
        current_messages)

    return {"messages": [response_message]}
```

Defines the LLM node

Gets the current messages from the agent state

Adds a system message to the current messages to set the behavior of the assistant

Invokes the LLM model with the current messages. The LLM will decide whether to call the search tool or return an answer.

Appends the system message to the current messages

Returns the response message, which contains the tool call or the answer

Restart your application in debug mode, and ask the following:

```
You: Suggest two Cornwall beach towns with nice weather
```

At your first breakpoint in `llm_node()`, examine `current_messages` before and after appending the system message. Then, check `response_message`—now you'll see the following tool call:

```
{'name': 'search_travel_info', 'args': {'query': 'beach towns in Cornwall'},
'id': 'call_rJrYwfFG4BwaUPWBaQeUrn7o', 'type': 'tool_call'}
```

What does this mean? The LLM is now forced to use your semantic search tool for candidate beach towns and won't pick them from its own "knowledge." This minimizes hallucinations and guarantees the data used comes from your knowledge base.

Continue stepping through the code (press F5). Right before you submit the `current_messages` to the LLM again, your messages will look like this:

```
[HumanMessage(content='Suggest 2 Cornwall beach towns with nice weather',
additional_kwargs={}, response_metadata={}),
SystemMessage(content='You are a helpful assistant that can search travel
information and get the weather forecast. Only use the tools to find the
information you need (including town names).', additional_kwargs={},
response_metadata={}),
AIMessage(content=[], ..., tool_calls=[{'name': 'search_travel_info', 'args':
{'query': 'beach towns in Cornwall'}, 'id': 'call_rJrYwfFG4BwaUPWBaQeUrn7o',
'type': 'tool_call'}], ...),
ToolMessage(content='"<p id=\\"mwrg\\">Cornwall, in particular Newquay, is
the UK\'s <b id=\\"mwrw\\"><a rel=\\"mw:WikiLink\\" href=\\"//
```

```
en.wikivoyage.org/wiki/Surfing\\" title=\\"Surfing\\" id=\\"mwsA\\">surfing</
a></b> capital, with equipment hire and surf schools present on many of the
county\'s beaches, and events like the UK championships or Boardmasters
festival.</p>\\n---\\n...</section><section ...', name='search_travel_info',
tool_call_id='call_rJrYwfFG4BwaUPWBaQeUrn7o'),
SystemMessage(content='You are a helpful assistant that can search travel
information and get the weather forecast. Only use the tools to find the
information you need (including town names).', additional_kwargs={},
response_metadata={})]
```

From the `ToolMessage`, the LLM now receives a list of possible beach towns from your vector store.

Next, as you step through and reach `llm_node()` again, the LLM will issue calls for the weather forecast in two of these towns:

```
AIMessage(content=[], ..., tool_calls=[
{'name': 'weather_forecast', 'args': {'town': 'Newquay'}, 'id':
'call_OoDb7UwfrWF8c79DrfK2w9mp', 'type': 'tool_call'},
{'name': 'weather_forecast', 'args': {'town': 'St Ives'}, 'id':
     'call_4Q4IaMIU9qO6sgfp4ls2Lwxw', 'type': 'tool_call'}], ...)
```

The towns selected by the LLM might differ in your run, but they'll always come from the results of your semantic search tool.

Step through the weather tool calls. Each `ToolMessage` you inspect should look like this:

```
ToolMessage(content='{"town": "Newquay", "weather": "foggy", "temperature":
20}', name='weather_forecast', tool_call_id='...')
ToolMessage(content='{"town": "St Ives", "weather": "windy", "temperature":
22}', name='weather_forecast', tool_call_id='...')
```

What if the weather isn't good? If the weather is less than ideal in those towns, continue to the next LLM response, and you'll see the following:

```
AIMessage(content=[], ...,
tool_calls=[
{'name': 'weather_forecast', 'args': {'town': 'Perranporth'}, 'id':
'call_LOqaMnreszfHb5vItFCapRSk', 'type': 'tool_call'},
{'name': 'weather_forecast', 'args': {'town': 'Falmouth'}, 'id':
     'call_iaOeeYAIxK1lyShjwPdLWQ6b', 'type': 'tool_call'}], ...)
```

Here, the LLM is asking for the weather in two more towns, likely because the previous results didn't meet the "nice weather" requirement. This process continues until the agent finds two towns with suitable conditions. After the final tool calls, the LLM generates a synthesized, fact-based answer:

```
AIMessage(content=[{'type': 'text', 'text': 'Two beach towns in Cornwall with
nice weather currently are:\n\n1. Perranporth - The weather is sunny with a
temperature of 31°C.\n2. Falmouth - The weather is windy but still warm with
a temperature of 28°C.\n\nWould you like more information about these towns
or other beach towns in Cornwall?', 'annotations': []}]
```

This appears to the user as

```
UK Travel Assistant (type 'exit' to quit)
You: Suggest two Cornwall beach towns with nice weather
Assistant: [{'type': 'text', 'text': 'Two beach towns in Cornwall with nice
weather currently are:\n\n1. Perranporth - The weather is sunny with a
temperature of 31°C.\n2. Falmouth - The weather is windy but still warm with
a temperature of 28°C.\n\nWould you like more information about these towns
or other beach towns in Cornwall?', 'annotations': []}]
```

In summary, by enhancing tool descriptions and providing explicit instructions via system prompts, you can guide the LLM to chain tool use in a multi-step, fact-grounded workflow—first searching for beach towns, then filtering by real-time weather. This produces answers that are both dynamic and reliable.

> **EXERCISE** Try replacing the mock weather tool with a real API, such as `OpenWeatherMap`, using LangChain's `OpenWeatherMap` integration. This will make your agent truly real time!

11.9 Using prebuilt components for rapid development

Up to this point, you've built an agent from the ground up: wiring together the graph, orchestrating tool calls, and stepping through every detail in the debugger. You now have a solid grasp of how tool calling works under the hood.

However, for most production scenarios, you'll want to reduce boilerplate and move faster—while still retaining transparency and observability when needed. The LangGraph library provides prebuilt agent components, such as the ReAct agent, that encapsulate much of this orchestration logic for you. Now let's see how dramatically you can simplify your agent by switching to a prebuilt approach.

11.9.1 Refactoring to use the LangGraph ReAct agent

Start by copying your previous script (`main_02_02.py`) to a new file: `main_03_01.py`. This refactoring not only removes low-level orchestration code but also ensures that your agent follows a proven, well-tested interaction pattern designed specifically for reliable tool use.

IMPORTING THE REMAININGSTEPS UTILITY

At the top of your script, add the following import:

```
from langgraph.managed.is_last_step import RemainingSteps
```

REMOVING MANUAL TOOL BINDING

You can now delete the line where you bound tools to the LLM:

```
llm_with_tools = llm_model.bind_tools(TOOLS)
```

The prebuilt agent will handle this for you internally.

UPDATING THE AGENTSTATE

Modify your `AgentState` definition to include a `remaining_steps` field. This field allows the agent to manage how many tool-calling rounds are left in a controlled way:

```
class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
    remaining_steps: RemainingSteps
```

REMOVING NODE AND GRAPH CONSTRUCTION

Now for the biggest simplification: delete your custom `ToolsExecutionNode` and `llm_node`, as well as any explicit graph wiring code. Replace it all with a single instantiation of the built-in LangGraph ReAct agent:

```
travel_info_agent = create_react_agent(
    model=llm_model,
    tools=TOOLS,
    state_schema=AgentState,
    prompt="""You are a helpful assistant that
can search travel information and get the weather forecast.
Only use the tools to find the information you need
(including town names).""")
```

That's it! The ReAct agent now orchestrates the flow, tool calling, and synthesis for you.

11.9.2 Running the prebuilt agent

When you run `main_03_01.py` and ask your usual test question

```
You: Suggest two Cornwall beach towns with nice weather
Assistant: [{'type': 'text', 'text': 'Two beach towns in Cornwall with nice
weather are:\n\n1. St Ives - It has sunny weather with a temperature around
26°C.\n2. Newquay - It also enjoys sunny weather with a temperature around
22°C.\n\nBoth towns are popular for their beautiful beaches and pleasant
weather.', 'annotations': []}]
```

you get the correct, grounded answer—with much less code.

11.9.3 Observing and debugging with LangSmith

A common concern when switching to high-level abstractions is loss of visibility: How do you know the agent is actually following the right reasoning steps? While you can still debug tool functions directly, the flow inside the agent itself is less exposed. This is where LangSmith comes in. LangSmith enables full tracing and inspection of agent behavior, including tool calls, LLM reasoning, and intermediate states.

11.9.4 Enabling LangSmith tracing

To enable tracing, add the following lines to your `.env` file:

```
LANGSMITH_TRACING=true
LANGSMITH_ENDPOINT="https://api.smith.langchain.com"
```

```
LANGSMITH_API_KEY="<your-langsmith-api-key>"
LANGSMITH_PROJECT="langchain-in-action-react-agent"
```

After rerunning your application and submitting a question, you can log into LangSmith (https://smith.langchain.com):

1. Select Tracing Projects in the Observability menu (left-hand sidebar).
2. Click your project (`langchain-in-action-react-agent`) in the right-hand panel.
3. Open the latest trace.

You'll see the full execution trace, as shown in figure 11.4.

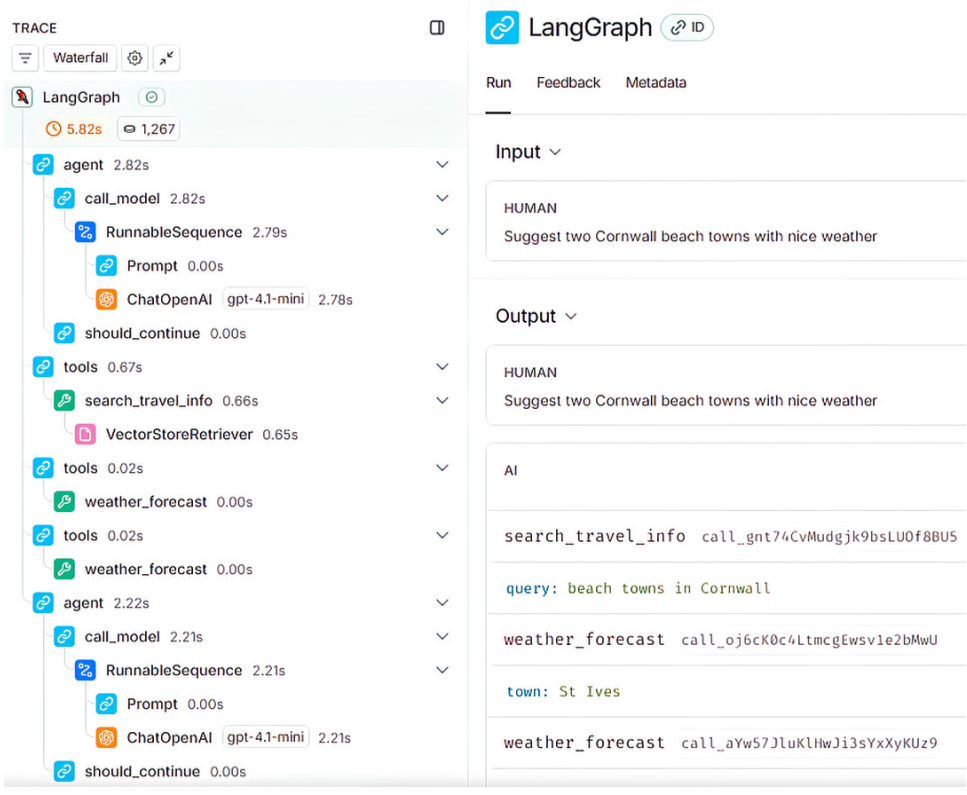

Figure 11.4 LangSmith agent execution trace. This trace visualizes each step of the agent's workflow—including LLM calls, tool invocations, and message flow—when answering the prompt, "Suggest two Cornwall beach towns with nice weather."

The LangSmith graphical trace shows every tool call, every LLM step, and the flow of messages—so even when you use a prebuilt agent, you retain the ability to audit, debug, and understand exactly what the agent is doing at every stage.

In summary, by combining prebuilt LangGraph agent components with LangSmith for observability, you can quickly build robust, production-ready agentic applications—while still maintaining full transparency and control when needed. This workflow has become the foundation for many modern AI agent systems in real-world use today.

Summary

- LangGraph builds agents using node-based architecture (explicit graphs with hardcoded conditional routing) or ReAct agents (built-in reasoning-action loops with automatic tool selection).
- Tool registration defines Python functions with type hints and docstrings that LLMs can discover and invoke. The docstring becomes the tool description that guides LLM selection.
- The LLM reads tool descriptions and decides which to call based on the user's question. Write clear, specific tool descriptions that explain when to use each tool and what inputs it expects.
- State inspection reveals which tool the LLM chose, what arguments it passed, what the tool returned, and how the agent used that output to continue or conclude. This aids in debugging and refinement.
- Tool descriptions must specify input parameters, expected outputs, and when to use the tool. For example: `"search_customer(name: str) -> dict: Finds customer records by name. Use when user asks about specific customer details."`
- Multi-tool workflows chain outputs across tools with the LLM orchestrating the sequence. For example, `search_customer → get_orders → calculate_total` happens automatically based on the user's request.
- The LLM manages the sequence without hardcoded logic. Trust the LLM to chain tools appropriately by providing clear tool descriptions and validating outputs through testing.
- The ReAct agent in LangGraph provides built-in tool calling, error management, and retry logic. This simplifies agent creation compared to manually building graphs with conditional edges.
- LangSmith traces capture the full execution flow. They show LLM reasoning steps, tool selection decisions, intermediate outputs, and final responses in a visual timeline.
- Each trace includes token counts, latency, and error states for debugging failed agent runs.
- Tool-calling agents form the foundation for multi-agent systems. Specialized agents (researcher, writer, reviewer) each have distinct tool sets and are coordinated by supervisor agents.
- Tool functions should return structured data (dictionaries, lists) not strings when possible. Structured returns enable the LLM to extract specific fields rather than parsing unstructured text.

- Tool selection depends on description quality. Test descriptions by running sample queries and checking if the agent selects the correct tool. Revise descriptions that cause selection errors.
- Implement custom error handling in tools by wrapping tool logic in `try-except` and returning structured error messages such as `{"error": "Customer not found", "details": "..."}`.